# 1.REAL-TIME CHAT APPLICATION

## Using Node.js, Express.js, MongoDB, Mongoose and Socket.IO

---

## 2. Aim

**To develop a real-time chat application using:**

- Node.js
- Express.js
- MongoDB
- Mongoose
- Socket.IO
- HTML, CSS and JavaScript
- Visual Studio Code

The application allows multiple users to exchange messages instantly without refreshing the webpage. Messages are also stored in MongoDB.

---

# 3. Software Requirements

Install the following software:

| Software | Purpose |
|---|---|
| Node.js | Runs JavaScript on the server |
| npm | Installs Node.js packages |
| Visual Studio Code | Code editor |
| MongoDB Atlas | Cloud MongoDB database |

| | |
|---|---|
| Web Browser | Runs the chat application |

### Official websites

Node.js:

https://nodejs.org/

MongoDB Atlas:

https://www.mongodb.com/atlas

Visual Studio Code:

https://code.visualstudio.com/

MongoDB Atlas provides the database in the cloud, so students do not need to install MongoDB Server locally.

---

# 4. Theory

A real-time chat application allows users to exchange messages immediately without refreshing the webpage.

The technologies used in this application have different responsibilities.

### Node.js

Node.js provides the runtime environment for executing JavaScript on the server.

### Express.js

Express.js is used to create the web server and serve the frontend files.

### MongoDB

MongoDB is a NoSQL database used to permanently store chat messages.

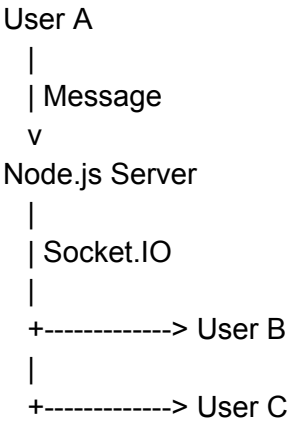
### Mongoose

Mongoose provides an easy way for Node.js applications to work with MongoDB. It allows us to define schemas and models.

### Socket.IO

Socket.IO provides real-time, bidirectional communication between the browser and server.

For example:

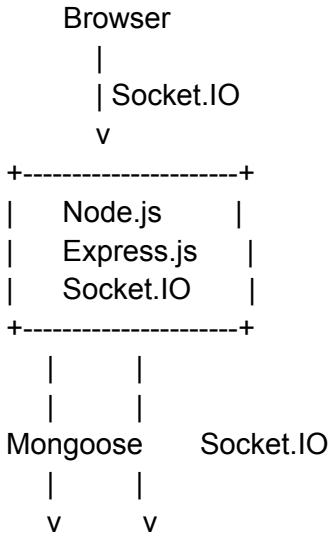



At the same time, the message is stored in MongoDB.

---

# 5. Application Architecture

The application works as follows:



```
+---------+   Other Users
| MongoDB |
+---------+
```

The process is:

```
User enters message
      ↓
Browser sends message using Socket.IO
      ↓
Node.js server receives message
      ↓
Mongoose saves message
      ↓
MongoDB stores message
      ↓
Socket.IO broadcasts message
      ↓
All connected users receive message
```

---

# 6. Step 1 — Check Node.js Installation

Open **Command Prompt**, **PowerShell**, or the **VS Code Terminal**.

Run:

```
node --version
```

Then:

```
npm --version
```

You should see version numbers.

Example:

```
v22.x.x
10.x.x
```

The exact version may be different.

If Node.js is not installed, download it from:

https://nodejs.org/

---

# 7. Step 2 — Create the Project Folder

Create a folder named:

RealTimeChat

For example:

C:\Users\Student\Desktop\RealTimeChat

Open this folder using Visual Studio Code.

In VS Code:

File
↓
Open Folder
↓
Select RealTimeChat

---

# 8. Step 3 — Open the Terminal

In Visual Studio Code, select:

Terminal → New Terminal

Make sure the terminal is inside the project folder.

Example:

PS C:\Users\Student\Desktop\RealTimeChat>

---

# 9. Step 4 — Initialize the Node.js Project

Run:

npm init -y

This creates:

package.json

The `package.json` file contains information about the Node.js project and its dependencies.

---

# 10. Step 5 — Install Required Packages

Run:

npm install express mongoose socket.io

After installation, the project will contain:

node_modules
package.json
package-lock.json

The three packages are:

### Express

express

Used to create the web server.

### Mongoose

mongoose

Used to connect Node.js to MongoDB.

### Socket.IO

socket.io

Used for real-time communication.

---

# 11. Step 6 — Create the Project Structure

Inside the `RealTimeChat` folder, create:

```
RealTimeChat/
│
├── node_modules/
│
├── public/
│   └── index.html
│
├── server.js
│
├── package.json
│
└── package-lock.json
```

The `public` folder contains the frontend.

The `server.js` file contains the backend.

---

# 12. Step 7 — Create a MongoDB Atlas Account

Open:

https://www.mongodb.com/atlas

Create an account or sign in.

MongoDB Atlas is MongoDB's cloud database service.

---

# 13. Step 8 — Create a MongoDB Cluster

After logging in:

1. Create a project.
2. Create a database deployment/cluster.
3. Choose a free option if available.
4. Give the deployment a suitable name.

For example:

ChatCluster

The exact Atlas interface may change over time.

---

# 14. Step 9 — Create a MongoDB Database User

In MongoDB Atlas, go to:

Security
↓
Database Access

Create a database user.

Example:

Username:
chatuser

Password:
YourOwnPassword

### Important

The database username/password is different from your MongoDB Atlas website login.

The application must use the **database user's username and password**.

MongoDB documents that Atlas database users are separate from Atlas users and are required for application access to the cluster.

---

# 15. Step 10 — Configure Network Access

In MongoDB Atlas, go to:

Security
↓
Network Access

Add the IP address of the computer from which you will run the application.

Choose:

Add My Current IP Address

MongoDB Atlas requires the connecting IP address to be present in the project's IP access list.

For a classroom/lab environment, the instructor may configure the appropriate network access policy.

For production applications, do not unnecessarily allow access from every IP address.

---

# 16. Step 11 — Get the MongoDB Connection String

In MongoDB Atlas:

Database
↓
Your Cluster
↓
Connect
↓
Drivers

Select:

Node.js

Copy the connection string.

It will look similar to:

mongodb+srv://<username>:<password>@<cluster>.mongodb.net/?retryWrites=true&w=majority

MongoDB Atlas provides the connection string from the Connect → Drivers section.

---

# 17. IMPORTANT — MongoDB Password

If your MongoDB password contains special characters such as:

@
#
$
%
:
/
?

the characters may need to be URL-encoded when they appear inside the connection string.

For example:

@

becomes:

%40

Therefore:

MyPassword@123

becomes:

MyPassword%40123

MongoDB specifically documents this requirement for reserved characters in connection strings.

A simple option for students during this practical is to create a database password that does not contain special URL characters.

**Never share your real database password with classmates or in a public repository.**

---

# 18. Step 12 — Create `server.js`

Inside the `RealTimeChat` folder, create:

server.js

Add the following code:

```
const express = require("express");
const http = require("http");
const mongoose = require("mongoose");
```

```
const { Server } = require("socket.io");

const app = express();
const server = http.createServer(app);
const io = new Server(server);


// ========================================
// MongoDB Connection
// ========================================

const MONGO_URI =

"mongodb+srv://YOUR_USERNAME:YOUR_PASSWORD@YOUR_CLUSTER.mongodb.net/ch
atDB?retryWrites=true&w=majority";


// ========================================
// Message Schema
// ========================================

const messageSchema = new mongoose.Schema({

   username: {
      type: String,
      required: true
   },

   message: {
      type: String,
      required: true
   },

   createdAt: {
      type: Date,
      default: Date.now
   }

});


// ========================================
// Message Model
// ========================================
```

```
const Message = mongoose.model("Message", messageSchema);


// ========================================
// Serve Frontend
// ========================================

app.use(express.static("public"));


// Home Route
app.get("/", (req, res) => {

    res.sendFile(__dirname + "/public/index.html");

});


// ========================================
// Start Application
// ========================================

async function startServer() {

    try {

        // Connect to MongoDB
        await mongoose.connect(MONGO_URI);

        console.log("MongoDB Connected Successfully");


        // ========================================
        // Socket.IO Connection
        // ========================================

        io.on("connection", (socket) => {

            console.log("User Connected:", socket.id);


            // ========================================
            // Load Previous Messages
```

```
// ========================================

Message.find()
    .sort({ createdAt: 1 })
    .limit(100)
    .then((messages) => {

        socket.emit("previous messages", messages);

    })
    .catch((error) => {

        console.error(
            "Error loading messages:",
            error
        );

    });


// ========================================
// Receive Chat Message
// ========================================

socket.on("chat message", async (data) => {

    try {

        const newMessage = new Message({

            username: data.username,

            message: data.message

        });


        // Save message to MongoDB
        const savedMessage =
            await newMessage.save();


        // Send message to all users
        io.emit(
```

```
            "chat message",
            savedMessage
        );


    } catch (error) {

        console.error(
            "Error saving message:",
            error
        );

    }

});


// ========================================
// User Disconnected
// ========================================

socket.on("disconnect", () => {

    console.log(
        "User Disconnected:",
        socket.id
    );

});

});


// ========================================
// Start HTTP Server
// ========================================

const PORT = 3000;

server.listen(PORT, () => {

    console.log(
        `Server Running at http://localhost:${PORT}`
    );
```

```
        });

    } catch (error) {

        console.error(
            "MongoDB Connection Failed:"
        );

        console.error(error.message);

    }

}


startServer();
```

---

# 19. Step 13 — Modify the MongoDB Connection String

Find:

```
const MONGO_URI =

"mongodb+srv://YOUR_USERNAME:YOUR_PASSWORD@YOUR_CLUSTER.mongodb.net/chatDB?retryWrites=true&w=majority";
```

Replace:

YOUR_USERNAME

with your MongoDB database username.

Replace:

YOUR_PASSWORD

with your MongoDB database password.

Replace:

YOUR_CLUSTER

with the cluster hostname provided by MongoDB Atlas.

For example, the structure will be:

const MONGO_URI =

"mongodb+srv://studentuser:StudentPassword2026@cluster0.xxxxx.mongodb.net/chatDB?retryWrites=true&w=majority";

Do not copy the example credentials.

Use your own MongoDB credentials.

---

# 20. Step 14 — Create `index.html`

Inside the `public` folder create:

index.html

Add:

```
<!DOCTYPE html>
<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >
```

```
<title>Real Time Chat Application</title>

<style>

  * {
    box-sizing: border-box;
  }

  body {

    margin: 0;

    padding: 0;

    font-family: Arial, sans-serif;

    background: #f2f2f2;

  }

  .container {

    width: 90%;

    max-width: 700px;

    margin: 40px auto;

    background: white;

    padding: 20px;

    border-radius: 10px;

    box-shadow:
      0 0 10px
      rgba(0, 0, 0, 0.15);

  }
```

```
h2 {
    text-align: center;
    color: #333;
}

.username-section {
    margin-bottom: 15px;
}

#username {
    width: 100%;
    padding: 12px;
    border: 1px solid #ccc;
    border-radius: 5px;
}

#messages {
    list-style: none;
    padding: 0;
    height: 400px;
    overflow-y: auto;
    border: 1px solid #ddd;
    border-radius: 5px;
    margin-bottom: 15px;
```

```
}

#messages li {
    padding: 10px;
    border-bottom: 1px solid #eee;
}

.message-username {
    font-weight: bold;
    color: #007bff;
}

.message-text {
    color: #333;
}

.message-time {
    font-size: 11px;
    color: #888;
}

.input-section {
    display: flex;
    gap: 10px;
```

```
        }

        #message {
            flex: 1;
            padding: 12px;
            border: 1px solid #ccc;
            border-radius: 5px;
        }

        button {
            padding: 12px 20px;
            background: #007bff;
            color: white;
            border: none;
            border-radius: 5px;
            cursor: pointer;
        }

        button:hover {
            background: #0056b3;
        }
    </style>
</head>
```

```
<body>


    <div class="container">

        <h2>
            Real Time Chat Application
        </h2>


        <!-- Username -->

        <div class="username-section">

            <input
                type="text"
                id="username"
                placeholder="Enter your username"
            >

        </div>


        <!-- Messages -->

        <ul id="messages"></ul>


        <!-- Message Input -->

        <div class="input-section">

            <input
                type="text"
                id="message"
                placeholder="Enter your message"
            >

            <button onclick="sendMessage()">
                Send
            </button>

        </div>
```

```
</div>


<!-- Socket.IO Client -->

<script src="/socket.io/socket.io.js"></script>


<script>

    // Connect to server

    const socket = io();


    // Get HTML elements

    const usernameInput =
        document.getElementById("username");

    const messageInput =
        document.getElementById("message");

    const messagesList =
        document.getElementById("messages");


    // ========================================
    // Send Message
    // ========================================

    function sendMessage() {

        const username =
            usernameInput.value.trim();

        const message =
            messageInput.value.trim();


        // Check username

        if (username === "") {
```

```
        alert(
            "Please enter your username"
        );

        return;

    }


    // Check message

    if (message === "") {

        return;

    }


    // Send message to server

    socket.emit(
        "chat message",
        {

            username: username,

            message: message

        }
    );


    // Clear input

    messageInput.value = "";

    messageInput.focus();

}


// ======================================
// Display Message
// ======================================
```

```
function displayMessage(data) {

    const li =
        document.createElement("li");


    const username =
        document.createElement("span");

    username.className =
        "message-username";

    username.textContent =
        data.username + ": ";


    const message =
        document.createElement("span");

    message.className =
        "message-text";

    message.textContent =
        data.message;


    const time =
        document.createElement("div");

    time.className =
        "message-time";


    if (data.createdAt) {

        const date =
            new Date(data.createdAt);

        time.textContent =
            date.toLocaleString();

    }
```

```
    li.appendChild(username);

    li.appendChild(message);

    li.appendChild(time);


    messagesList.appendChild(li);


    // Scroll to latest message

    messagesList.scrollTop =
        messagesList.scrollHeight;

}


// ======================================
// Receive New Message
// ======================================

socket.on(
    "chat message",
    function(data) {

        displayMessage(data);

    }
);


// ======================================
// Receive Previous Messages
// ======================================

socket.on(
    "previous messages",
    function(messages) {

        messages.forEach(
            function(data) {
```

```
                displayMessage(data);

            }
        );

    }
);


// ========================================
// Press Enter to Send
// ========================================

messageInput.addEventListener(
    "keypress",
    function(event) {

        if (event.key === "Enter") {

            sendMessage();

        }

    }
);

</script>


</body>

</html>
```

---

# 21. Step 15 — Verify the Project Structure

Your project should now look like:

```
RealTimeChat/
│
├── node_modules/
│
```

```
├── public/
│   └── index.html
│
├── server.js
│
├── package.json
│
└── package-lock.json
```

Do not create another `index.html` outside the `public` folder.

---

# 22. Step 16 — Start the Server

Open the VS Code terminal.

Make sure you are inside:

RealTimeChat

Run:

node server.js

If everything is correct, you should see:

MongoDB Connected Successfully
Server Running at http://localhost:3000

---

# 23. Step 17 — Open the Application

Open a browser.

Go to:

http://localhost:3000

The chat application should appear.

---

# 24. Step 18 — Test the Application

Enter a username.

Example:

Alice

Enter a message:

Hello everyone!

Click:

Send

The message should appear immediately.

---

# 25. Step 19 — Test Real-Time Communication

This step is important.

Open the application in **two browser tabs**.

### Browser Tab 1

Enter:

Username: Alice
Message: Hello Bob

Click Send.

### Browser Tab 2

Enter:

Username: Bob
Message: Hello Alice

Click Send.

Both users should see both messages.

No page refresh should be required.

This demonstrates Socket.IO real-time communication.

---

# 26. Step 20 — Test MongoDB Storage

Send several messages.

For example:

Alice: Hello
Bob: Hi Alice
Alice: How are you?
Bob: I am fine.

Then open MongoDB Atlas.

Go to:

Database
↓
Browse Collections

You should find the database:

chatDB

and a collection similar to:

messages

The documents will contain information similar to:

```
{
   "_id": "...",
   "username": "Alice",
   "message": "Hello",
   "createdAt": "..."
}
```

This proves that the chat messages are being stored in MongoDB.

---

# 27. Step 21 — Test Previous Messages

Close the browser tab.

Open:

http://localhost:3000

again.

The previously stored messages should be loaded from MongoDB.

This happens because the server executes:

```
Message.find()
   .sort({ createdAt: 1 })
   .limit(100)
```

and sends those messages to the newly connected client.

---